Secure Task By Akuphe

YouTube video script

16 October 2026 | Complete tutorial script | AWS DevSecOps

A complete recording script from introduction to closing, with architecture cues, a local demo, infrastructure and source fixes, staging review, production approval, and teardown.

Narration / **Screen directions** / **Commands** / **Pause cues**

01 Introduction

CAMERA *Begin with the problem before the channel greeting. Briefly show the application, a blocked infrastructure run, and two source findings in DefectDojo.*

Generative AI has made it much easier to turn an idea into a working web application. We can build a frontend, connect an API, and add a database quickly.

But how do we decide that application is ready for production?

PAUSE *Let the question land before continuing.*

A working homepage tells us very little about exposed secrets, unsafe queries, vulnerable packages, or the security of the infrastructure underneath it.

In this tutorial, we are going to make those checks part of the delivery process.

Welcome to the channel. I'm Akuphe, and today we are building an end-to-end DevSecOps project on AWS using GitHub Actions, Terraform, Docker, and open-source security tools.

We will run the application locally, let security checks stop our infrastructure deployment, fix the configuration, and then investigate two source-code findings. After the fixes pass, we will build and scan the image, deploy to staging, inspect it, and approve production.

Let's start with the architecture so you can see how the entire project fits together.

02 Project rundown with the architecture

SCREEN *Keep the architecture diagram visible throughout this section. Follow the flow from GitHub and Terraform to the scanners, ECR, staging, and production. Highlight each AWS service as it is named.*

Our application is Secure Task By Akuphe. It is a task management application with a React frontend, an Express backend, and a PostgreSQL database. We can register an account, create a project, add tasks, and update their status.

The source code lives in GitHub. GitHub Actions runs our workflows, and we trigger each top-level workflow manually so we can explain and inspect every stage during the tutorial.

Terraform defines the AWS infrastructure in us-east-1. Checkov examines that configuration before the workflow authenticates to AWS or creates the Terraform state bucket. We also check the resolved platform plan before apply.

Amazon S3 stores the encrypted, versioned Terraform state. The VPC provides public and private subnets, routing, and security groups. A NAT gateway gives resources in private subnets an outbound route for downloads and updates.

Amazon ECR stores the application image after it passes the image security policy. Amazon ECS with Fargate runs that image in private application tasks, and Amazon RDS provides PostgreSQL for staging and production.

Each environment has an Application Load Balancer with an AWS-generated URL. We will use those endpoints to inspect the application without setting up a custom domain.

IAM controls permissions. Secrets Manager stores private credentials, and CloudWatch collects application logs and monitoring information.

DefectDojo Community Edition runs on a private EC2 instance. Systems Manager provides the encrypted connection we use to reach its dashboard and upload release findings.

The application is deployed to staging first. Once the automated checks pass, we inspect staging ourselves. A required approval then allows the same image digest to move to production.

That is the delivery path. Now let's look at the security checks and the four issues we will fix along the way.

03 Security tools and the four issues

SCREEN *Show the seven scanner logos. Keep DefectDojo visually separate as the findings platform.*

We are using seven security tools, and each has a specific job.

Semgrep checks source code for potentially unsafe patterns. We use it to identify our SQL injection and unsafe eval examples.

Gitleaks looks for exposed credentials in Git history. Removing a secret from the latest file does not remove it from an earlier commit.

OSV-Scanner checks dependency versions in the lockfile against known vulnerability data.

Checkov examines Terraform for infrastructure misconfigurations. Our two custom policies require S3 public-access protections and RDS storage encryption.

Hadolint checks the Dockerfile and its shell instructions for problems and build practices that need attention.

Trivy scans the built container image for known vulnerabilities in operating-system and application packages. It runs before the image is published to ECR.

ZAP runs against staging. We use its baseline scan to inspect reachable unauthenticated pages and responses. Logged-in workflows need additional testing beyond that baseline.

SCREEN *Show four issue labels grouped by stage: infrastructure before provisioning, source code before the image build.*

Our first two issues are in the real Terraform configuration: one S3 public-access protection is disabled, and RDS storage encryption is disabled. These must be corrected before infrastructure provisioning can continue.

The other two are controlled source-code exercises: an interpolated SQL query and unsafe use of eval. They are excluded from the deployed application image, but the release scan includes them for our demonstration.

We will show the infrastructure findings in GitHub Actions, then show the source findings and their remediation history in DefectDojo.

The most important lesson is to understand the security concept behind each tool: what risk it checks for, where it belongs, and how its result should affect a release.

You can choose other tools for the same categories. Opengrep is another source-analysis option, TruffleHog detects secrets, KICS checks infrastructure, and Grype scans container packages. When replacing a scanner, verify its coverage, report format, failure behavior, and integration with your pipeline.

04 Run the application locally and prepare GitHub

SCREEN *Start Docker Desktop. Open a terminal in the project root, where the application docker-compose.yml is located. Run only the commands below, without copying step numbers.*

Before we provision AWS, let's run the application locally and check the main features. Docker Compose will build the application and start it together with PostgreSQL.

```
docker compose up --build -d
docker compose ps
curl --fail http://127.0.0.1:3000/health/ready
```

SCREEN *Wait for startup. Repeat the readiness check if needed. Show the expected response {"status":"ready"}, then open http://127.0.0.1:3000/.*

The readiness response confirms the backend can reach the database. Now I'll register a test account, create a project called DevSecOps Demo, and add a task called Verify local application.

SCREEN *Use a password of at least 12 characters and keep it masked. Create the project and task, update the task status, refresh, then sign out and back in.*

Our frontend, backend, and database are working together. We now have a functional baseline to compare with the application we will deploy to AWS. The local data stays on this computer.

SCREEN *Open GitHub and the IDE. Show the complete project, including .github/workflows, then the repository secret names. Keep all secret values hidden.*

For GitHub, we need an account and Git authenticated through our terminal or IDE. Create an empty repository, push the complete project to main, and follow the repository setup in the runbook.

Add the AWS access key ID, AWS secret access key, and the repository administration token as repository secrets. Temporary AWS credentials also need a session token. The runbook lists the exact names and permissions.

Pushing code saves a revision. It does not start these workflows. We will use Run workflow to choose when each run begins.

05 Block infrastructure provisioning and fix the settings

SCREEN *Open Actions > Provision AWS infrastructure > Run workflow. Select main, enter the AWS account ID, and start the run. Open Check Terraform before any AWS provisioning.*

This first run starts with two deliberately incorrect infrastructure settings. Checkov will scan the Terraform source, and the policy gate should block the workflow before AWS authentication or state-bucket bootstrap.

SCREEN *Wait for the failed gate. Show the custom policy IDs CKV2_AKUPHE_1 and CKV2_AKUPHE_2, the policy output, and the infrastructure-preflight-security artifact.*

The first issue is in the S3 public-access configuration. The Block Public Policy setting is false. That alone does not automatically make the bucket public, but it disables this bucket-level protection against public policies. Our policy requires all four public-access protections to stay enabled.

The second issue is RDS storage encryption. That setting is false, so the database configuration does not meet our encryption-at-rest requirement. This resource definition covers both staging and production.

On a first installation, DefectDojo has not been provisioned yet. That is why we are reviewing these findings here in GitHub Actions.

The provisioning jobs are skipped. Approval cannot override this failed security gate.

SCREEN *In infra/bootstrap/main.tf, find aws_s3_bucket_public_access_block.state and change the marked setting to true. Keep the other three protections enabled.*

```
block_public_policy = true
```

SCREEN *In infra/platform/main.tf, find aws_db_instance.main and change the marked setting to true.*

```
storage_encrypted = true
```

We fix the configuration and keep the checks enabled. Fixing only one setting leaves the other policy blocking.

Now I'll commit and push both corrections, then start a new provisioning run at the updated main commit. Re-running the old job would still check the old code.

06 Provision the platform and open DefectDojo

SCREEN *Start a new Provision AWS infrastructure run after pushing the fixes. Continue this narration only after the preflight gate passes.*

The infrastructure checks now pass. The workflow can continue to configure the approval environments and authenticate to the selected AWS account.

When planning pauses for approval, I'll review the run and approve the infrastructure environment. This permits the state-bucket bootstrap and platform planning steps.

The workflow checks whether the dedicated S3 state bucket exists. It creates it if needed, or verifies ownership and reuses it for this installation.

Terraform then produces the platform plan. Checkov scans that resolved plan with the custom policies as another check before apply. Once the policy passes, we review the plan and approve the waiting apply job.

SCREEN *Show the plan, its security reports, and the apply approval. Let provisioning and Dojo setup finish. Pause or cut the recording while waiting.*

After apply and Dojo setup complete, the workflow publishes the environment URLs and integration variables. The ECS application services are created by the later release, so an application URL can return a 503 response before that first release.

SCREEN *Follow Chapter 6 of the runbook to open the private Dojo dashboard through Session Manager. Retrieve the administrator credentials privately. Do not record secret values.*

DefectDojo is our place to review findings and their history. The scanners perform the checks, and the GitHub workflow enforces the policy.

We connect through Systems Manager because this Dojo instance is private. The workflow has already prepared the product, scanner tests, and separate integration account for reporting.

Now we can demonstrate the source-code findings.

07 Find the source issues and show their fixes

SCREEN *Confirm the two vulnerable scan-only source examples are prepared using the recording notes. Open Actions > Secure release > Run workflow. Select main and enable include_remediation_demo.*

I am enabling the remediation exercises so Semgrep scans the two source examples alongside the application. The other source scanners still run their normal checks.

SCREEN *After reporting completes, show the failed policy step and skipped Docker build. Open Secure Task By Akuphe > Secure Task By Akuphe remediation > semgrep in Dojo.*

The reports reach DefectDojo before the policy blocks the release. We can inspect the findings, while the release Docker build remains blocked.

Under the default policy, High and Critical findings block a release. Secrets, incomplete scans, and failed reporting also block.

The first example places input directly inside an SQL query string. That can let input change the meaning of the query. The fix is to keep the SQL text fixed and pass the search value as a separate parameter.

SCREEN *Replace the implementation in security/demo/unsafe-search.ts with the corrected version.*

```
export async function unsafeSearch(db: any, search: string) {
  return db.query("SELECT * FROM tasks WHERE title LIKE $1", [`%${search}%`]);
}
```

The second example passes input to eval, which treats it as JavaScript. This function only needs a number, so we can convert the input and reject a value that is not finite.

SCREEN *Replace the implementation in security/demo/unsafe-expression.ts with the corrected version.*

```
export function unsafeNumber(input: string): number {
  const value = Number(input);
  if (!Number.isFinite(value)) throw new Error("Enter a finite number");
  return value;
}
```

These are scan-only exercises. We keep them out of the deployed application image while using them to demonstrate detection, correction, and a clean rescan.

I'll keep both files present, commit the fixes, and push. Removing the files or disabling the exercise scan would not demonstrate that the code was corrected.

08 Rescan approve and publish the scanned image

SCREEN *Start a new Secure release run at the corrected main commit. Keep include_remediation_demo enabled.*

We are scanning the same files again at the new commit and reporting to the same Semgrep test in the remediation engagement.

When the completed reimport finishes, the previous exercise findings should become mitigated. Their history remains available, so we can show what was found and when it was fixed.

SCREEN *Show the two mitigated source findings and Import History. Confirm all blocking source conditions pass before continuing.*

Other tools can still report findings that need review. We are checking that these two exercise issues are resolved and that the release policy passes.

The workflow can now wait at the security-review approval. I'll review the reports and approve the image build.

SCREEN *Approve security-review. Show Build Docker image and scan with Trivy, then Report container findings and enforce image policy.*

Docker builds the release image from the reviewed commit. Trivy scans the operating-system and application packages inside it. Those results go to DefectDojo, and the image policy must pass before ECR publishing.

A blocking container finding stops the release before the push. If the image passes, the workflow publishes that exact scanned image to ECR without rebuilding it.

It uses a unique commit-and-run tag and records the image digest. That digest identifies the image content we will deploy and promote.

09 Inspect staging and approve production

SCREEN *Show the ECR image digest, then Deploy staging for inspection. Open the staging URL after deployment and health checks complete.*

ECS Fargate pulls the approved image from ECR and runs it in staging behind the load balancer. The workflow performs smoke checks, then runs the ZAP baseline scan and reports its results to DefectDojo.

Runtime reporting and policy must succeed before the staging review can complete.

SCREEN *Show the ZAP results and Staging ready for your review summary. Follow its staging link.*

Now I'll inspect the application myself. I'll register a staging test account, sign in, create a project, add a task, change its status, and refresh to check the saved result.

SCREEN *Perform the staging checks. Show the pending production approval after the checks pass.*

The automated checks have passed, and we have inspected the main user flow. I can now use Review deployments to approve the production environment.

If something is wrong in staging, we reject production and investigate before another release.

SCREEN *Approve production only after staging inspection. Wait for production deployment and smoke checks to complete, then open the production ALB URL.*

Production receives the same image digest tested in staging. I'll verify the production application at its own load balancer URL with a separate test account.

We have evidence from infrastructure checks, source checks, image scanning, runtime scanning, and human review. That evidence applies to the checks we configured; secure design and further testing remain part of the work.

10 Decommission and close the video

SCREEN *Record production verification first. When ready to remove the lab, open Decommission all project infrastructure, enter the AWS account ID and DESTROY securetask, review its inventory, and approve decommission.*

When we have finished the demonstration, we can remove the project through the separate decommission workflow. It reviews what will be deleted and waits for approval before the destructive stage.

This deletes the project's cloud infrastructure and data, so export anything you want to keep before approving it.

SCREEN *Wait for the workflow's verification result. Say the next line only if result.json confirms completed true and zero remaining-resource counts.*

The decommission result confirms the project resources have been removed. I will also remove the dedicated credentials when they are no longer needed.

CAMERA *Return to camera for the closing statement.*

Today we started with a working local application and built a delivery process around it.

We stopped two infrastructure misconfigurations before provisioning, fixed two source-code issues, tracked their remediation, scanned the container image, and tested staging before approving production.

The idea to take away is that every security check should have a clear purpose and a clear consequence. Understand the risks, review the evidence, fix the problems, and decide what is allowed to move forward.

You can use different tools and apply the same concepts in your own pipeline.

The project code and runbook are linked in the description so you can follow the walkthrough yourself. If this was useful, subscribe and let me know which security check you would add to this pipeline.

Thanks for watching. I'm Akuphe, and I'll see you in the next video.

Recording notes

Production guidance only. Do not read this section aloud.

Prepare the starting state

- Use a dedicated AWS sandbox and test data. Add your repository and runbook links to the video description before publishing.
- The two infrastructure settings in the tutorial source start as false. If they are already corrected in your recording branch, restore only the marked challenge settings before the first recorded infrastructure run. The provisioning gate must remain enabled. Do not apply the deliberately incorrect configuration manually.
- Your two source exercises may already be fixed. To show the initial findings, prepare the vulnerable implementations below before the first Secure release run. Keep their scan-only comments and Docker exclusions. Do not change the deployed application to add these vulnerabilities.

Original implementation for security/demo/unsafe-search.ts:

```
export async function unsafeSearch(db: any, search: string) {
  return db.query(`SELECT * FROM tasks WHERE title LIKE '%${search}%'`);
}
```

Original implementation for security/demo/unsafe-expression.ts:

```
export function unsafeNumber(input: string): number {
  return eval(input);
}
```

Recording checkpoints

- Keep the architecture visible during the project rundown. Highlight Checkov before AWS bootstrap, then the plan gate, source gate, image gate, staging, and production approval.
- Record a fresh workflow run after each pushed fix. Old runs use their original commit. Keep include_remediation_demo enabled for both source exercise runs.
- Four issue types do not mean exactly four report rows. RDS has two environments, and built-in Checkov rules can flag the same setting as a custom rule. New dependency findings may also appear.
- Initial infrastructure findings are in Actions artifacts because Dojo does not exist yet. Later release findings are uploaded to the tool-specific Dojo tests.
- Fix real blocking findings or scanner errors before continuing. Do not disable policies, reduce severities, skip scanners, or claim every tool has zero findings.
- All seven scanners must be visible in the explanation: Semgrep, Gitleaks, OSV-Scanner, Checkov, Hadolint, Trivy, and ZAP. DefectDojo is the findings platform. Opengrep, TruffleHog, KICS, and Grype are alternatives, not extra installed scanners.

Recording notes continued

Final recording checks

- The local Docker Compose build is a preview on your computer. The later security-review approval controls the CI release build, which is scanned before publication.
- Use the root Compose file for the local application. If automatic discovery fails, use docker compose -f /Users/akuphe/Desktop/securetask-devsecops/docker-compose.yml up --build -d on this Mac. The first build needs internet access; ports 3000 and 55432 must be available.
- Keep passwords, AWS keys, tokens, Secrets Manager values, and private .env files out of the recording. Use a password of at least 12 characters for application test accounts.
- Application ALB URLs use HTTP in this tutorial. The private Dojo connection uses encrypted Systems Manager transport. Do not describe all browser traffic as HTTPS.
- ZAP coverage is the configured unauthenticated baseline. Do not present it as a complete penetration test.
- Software scanners are open source; AWS hosting incurs charges. Skip long wait periods in the edit, and only announce success after the relevant verification succeeds.
- After recording, stop the local preview with docker compose down from the project root. This preserves its database volume. Close the Dojo tunnel with Ctrl+C. Review the decommission result before announcing that AWS cleanup is complete.

Reference links for the description

- **AWS S3 public-access protections**
- **AWS RDS encryption at rest**
- **Checkov custom YAML policies**
- **DefectDojo finding reimports**